WEB APPLICATION QA TEST REPORT

Realbugz – Online Store & Form Case

Supplementary checks: Conduit

Prepared by	Biswayan Banerjee
Test date	10–11 November 2026 (as displayed during testing)
Environment	Web browser on Windows desktop
Scope	Registration form, product discovery, cart, checkout, order management, and selected Conduit workflows

Executive summary

Testing covered core user journeys in the Realbugz practice application. Most observed functions behaved as expected, including product search, sorting, filtering, cart calculations, checkout required-field validation, order cancellation, and out-of-stock restrictions. The strongest issue observed was that checkout accepted a clearly nonsensical delivery address and created an order. A form-validation display inconsistency and repeat-submission behavior were also observed; the latter requires backend verification before it can be called a confirmed duplicate-record defect.

Important scope limitation: The assessment asks for at least five issues. The evidence collected in this session does not support five confirmed defects. This report deliberately does not invent findings. Additional targeted testing is required to meet that numerical requirement.

1. Test scope and approach

Applications exercised: Realbugz Case 02 (Form), Realbugz Case 05 (Online Store), and selected Conduit workflows. Testing was exploratory and manual. The tester used test account details and sample data, and observed visible browser behavior. No source-code inspection, database inspection, API verification, or payment processing was performed.

2. Test execution summary

Area / test	Observed result
Conduit registration, login, logout, article create/edit/delete	Passed in observed runs
Conduit favorites and comments (create/delete/empty/length/special characters)	Passed
Conduit profile update and invalid/empty field validation	Passed
Conduit article persistence	Initial disappearance could not be reproduced; a new article survived refresh and logout/login. Excluded as a defect.
Realbugz form: empty submission, invalid email, clear form, required fields	Required-field validation and clear-form behavior worked
Realbugz form: valid-format email	Submission succeeded, but the email field still appeared with a red outline before/at successful submission
Realbugz store: add/remove cart, quantity/subtotal, checkout required fields	Passed in observed runs
Realbugz store: sort, search, category/price filters, wishlist	Passed
Realbugz store: out-of-stock item and order cancellation	Passed
Realbugz store: invalid price range and no-result search	Passed

3. Defect and risk observations

Finding 1 – Checkout accepts a nonsensical delivery address

Steps to reproduce: Add a product to the cart; proceed to checkout; enter required checkout values, including a deliberately nonsensical address string; submit the order.

Expected: The checkout should validate that the delivery address meets basic completeness/format rules, or clearly indicate that address verification is not performed.

Actual: The order was placed successfully and the order summary displayed a long, random-looking address string.

Severity: Medium (provisional; depends on whether the store intentionally accepts free-form addresses and whether fulfilment validation occurs later).

Suspected cause: Client-side and/or server-side address validation may be missing or limited to non-empty checks. This is a hypothesis; backend behavior was not inspected.

Evidence: Order RB-10001 showed the success banner and the entered nonsensical address in the order details.

Finding 2 – Valid-format email retains invalid visual styling

Steps to reproduce: On the Realbugz Form case, trigger validation using an invalid email; replace it with qa.tester@example.com; observe the field styling and submit the otherwise completed form.

Expected: After a valid-format email is entered, the invalid-state styling should clear, or the form should explain any remaining reason for the error.

Actual: The email input remained outlined in red, but the application subsequently displayed “The form has been submitted!”

Severity: Low (visual feedback / validation-state inconsistency).

Suspected cause: The field's visual error state may not be refreshed when the input changes, even though submission validation accepts the new value.

Finding 3 – Form remains populated and can be submitted again

Steps to reproduce: Complete the form and submit it; dismiss the success alert; observe that the values remain; submit the unchanged form again.

Expected: After successful submission, the interface should make the completed state clear and prevent accidental repeat submission, or explicitly explain that another submission is allowed.

Actual: The success alert appeared again for the unchanged form data. The form remained populated after the first success.

Severity: Low / Medium (provisional).

Suspected cause: The form may not clear, lock, or track the completed submission state. It was not possible to verify whether duplicate records were actually created.

Verification note: This is a repeat-submission risk, not proof that duplicate database records exist. A database/admin check or unique submission identifier would be needed to confirm duplicate creation.

4. Root-cause analysis

The observed issues appear to involve validation and submission-state handling rather than a general failure of the shopping workflow. The checkout accepted a free-form address that was visibly nonsensical, suggesting that the application may validate only whether the field is populated rather than whether the address is meaningful. The email field retained red invalid styling even after a valid-format value was entered and the form accepted the submission, which suggests that the visual validation state may not be synchronized with the latest field value. The form also remained populated after a success alert and accepted another submission attempt. This could allow accidental repeat submissions if the backend does not enforce idempotency or uniqueness. These are hypotheses based on browser-visible behavior; confirming root causes would require inspecting server responses, stored records, or application code. The earlier Conduit article-disappearance observation was excluded because a later article remained visible after refresh and a fresh logout/login cycle.

5. Evidence screenshots

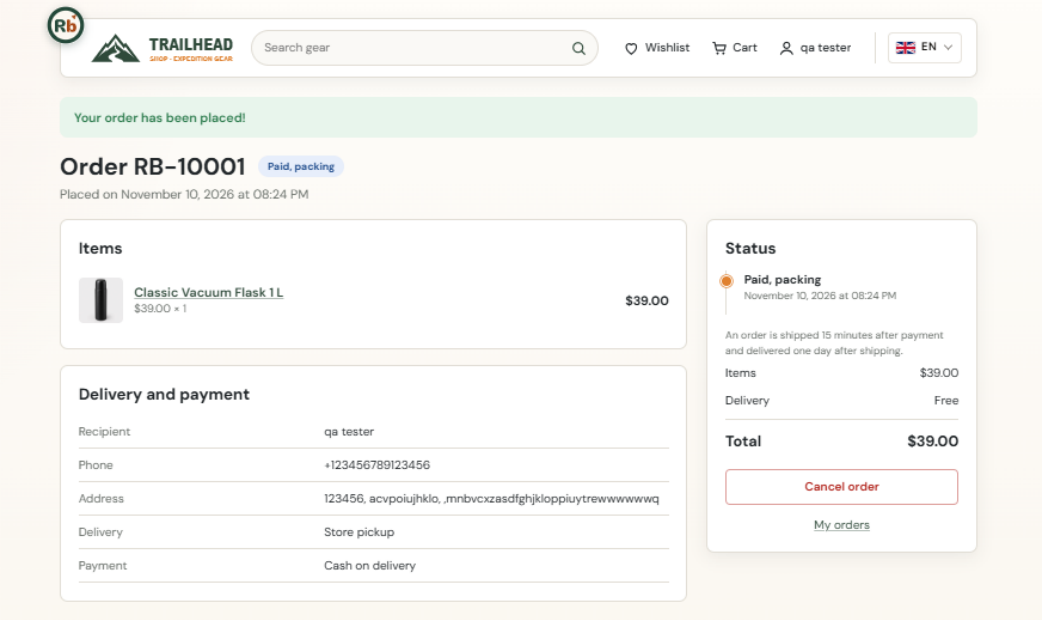


Figure 1. Order RB-10001 placed; order summary displays the supplied address.

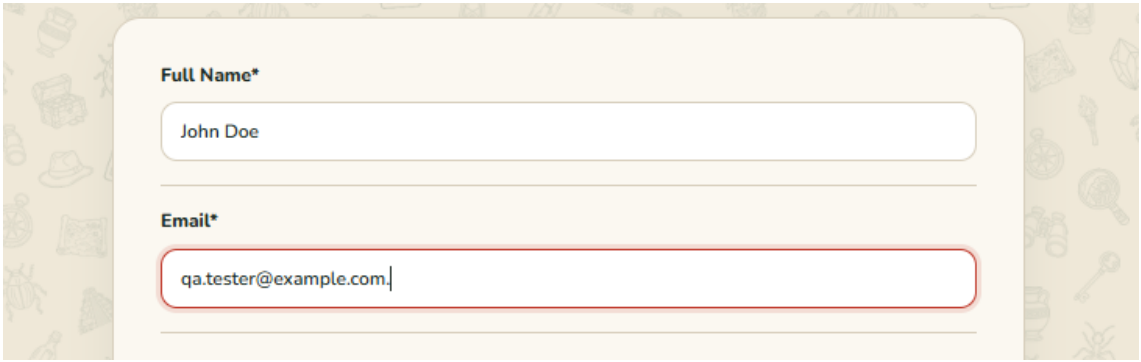


Figure 2. Valid-format email still displayed with red outline.

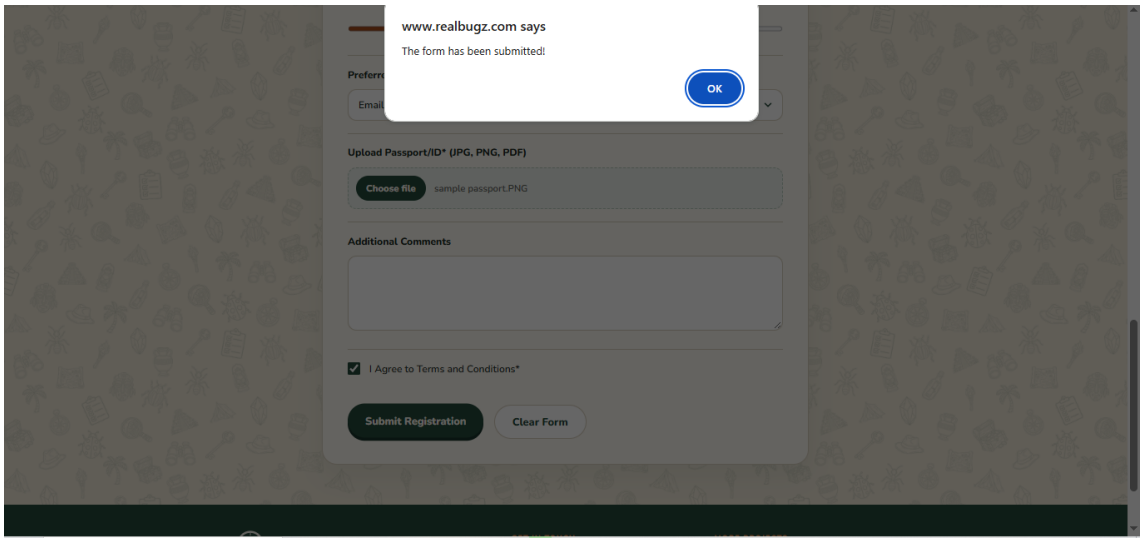


Figure 3. Form success alert: “The form has been submitted!”

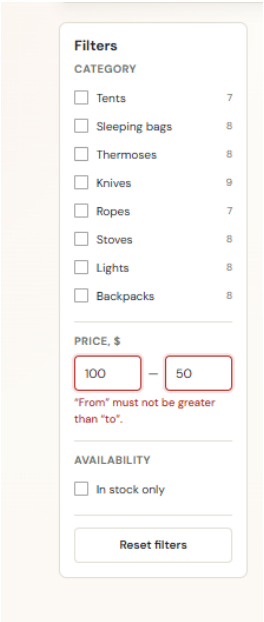


Figure 4. Invalid price range is rejected with a clear message.

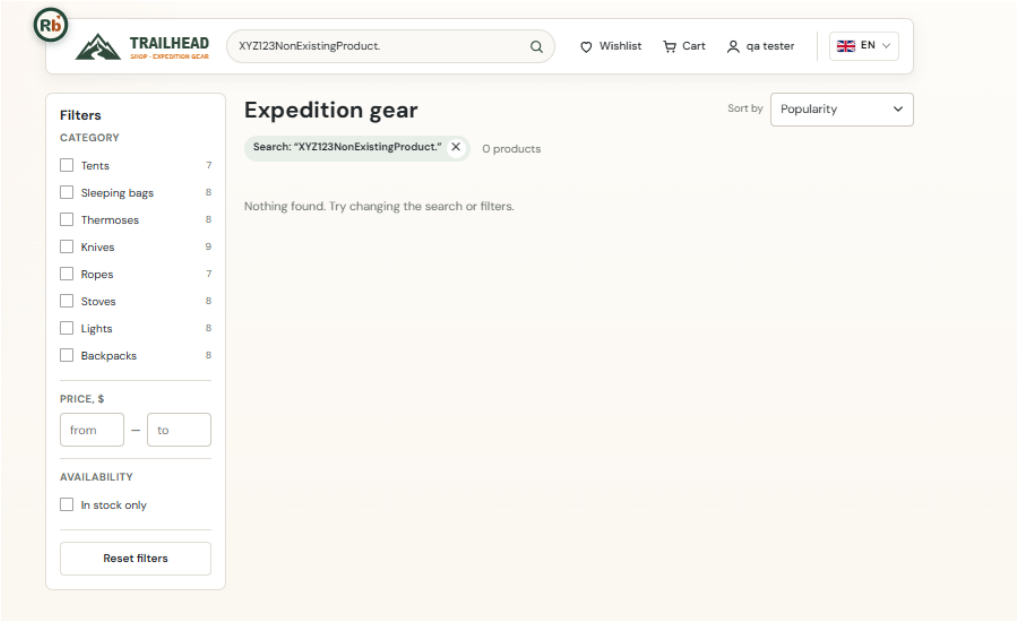


Figure 5. Non-existent product search returns zero products and a no-results message.

6. Conclusion and next steps

The tested happy paths and common validation paths were generally successful. Three observations are documented above, but only the address-validation behavior is a substantive functional concern based on the available evidence; the other two are lower-severity UI/submission-state concerns and require further verification. The requested minimum of five confirmed issues has not yet been met. To complete that requirement honestly, further focused testing is needed (for example, boundary values, duplicate account/order behavior, and server-side validation), with each issue reproduced and evidenced before inclusion.

Prepared from manual browser testing and screenshots supplied during the assessment. No claim is made that backend records, APIs, or payment systems were independently verified.